

Explain Dice Sum DP

The key to **LeetCode 1155 — Number of Dice Rolls With Target Sum** is to think:

"If I know the number of ways to make some sum using $n-1$ dice, how can I use one more die to reach target?"

This naturally leads to **Dynamic Programming**.

1. Understand the problem

You have:

- n dice
- Each die has values $1 \dots k$
- We need the number of ways to get exactly **target**

For example:

```
n = 2
k = 6
target = 7
```

Possible:

```
1 + 6
2 + 5
3 + 4
4 + 3
5 + 2
6 + 1
```

Answer = 6.

2. First think about brute force

Suppose:

```
n = 3
k = 6
target = 7
```

For every die, we can choose:

```
1, 2, 3, 4, 5, 6
```

So total possibilities:

```
6 × 6 × 6 = 216
```

We could generate every combination:

```
(1, 1, 1)
(1, 1, 2)
(1, 1, 3)
...
(6, 6, 6)
```

and count the ones whose sum is 7.

But when:

```
n = 30
k = 30
```

the total combinations are:

```
30^30
```

which is enormous.

So we need to avoid calculating the same things repeatedly.

3. The important observation

Suppose we're trying to make:

```
target = 7
```

using 2 dice.

Look at the **last die**.

It can be:

```
1
2
3
4
5
6
```

If the last die is 1, the previous dice must make:

```
7 - 1 = 6
```

If the last die is 2, the previous dice must make:

```
7 - 2 = 5
```

If the last die is 3:

```
7 - 3 = 4
```

and so on.

Therefore:

```
ways(2 dice, sum 7)
= ways(1 die, sum 6)
+ ways(1 die, sum 5)
+ ways(1 die, sum 4)
+ ways(1 die, sum 3)
+ ways(1 die, sum 2)
+ ways(1 die, sum 1)
```

This is the core recurrence.

4. Define the DP state

Let's define:

```
dp[dice][sum]
```

as:

Number of ways to obtain **sum** using exactly **dice** dice.

For example:

```
dp[2][7]
```

means:

Number of ways to get sum 7 using 2 dice.

Our answer is therefore:

```
dp[n][target]
```

5. How do we calculate a state?

Suppose we want:

```
dp[2][7]
```

The last die can have values from 1 to k.

For each possible last value **face**:

```
face = 1
remaining sum = 7 - 1 = 6
```

So:

```
dp[2][7] += dp[1][6]
```

Then:

```
face = 2
remaining sum = 7 - 2 = 5
```

So:

```
dp[2][7] += dp[1][5]
```

Continue:

```
dp[2][7] =
  dp[1][6]
+ dp[1][5]
+ dp[1][4]
+ dp[1][3]
+ dp[1][2]
+ dp[1][1]
```

Therefore the general formula is:

```
dp[dice][sum]
=
Σ dp[dice - 1][sum - face]
```

where:

```
face = 1 ... k
```

6. Base case

What happens with **zero dice**?

There is exactly **one way** to make sum **0**:

```
choose nothing
```

So:

```
dp[0][0] = 1
```

But:

```
dp[0][1] = 0
dp[0][2] = 0
...
```

This is extremely important.

Think of it as:

```
0 dice → sum 0
```

dp[0][0]=1

That **1** allows the first die to build all valid sums.

7. Let's build the table

Take:

```
n = 2
k = 6
target = 7
```

Initially:

```
dp[0][0] = 1
```

One die

One die can make:

1 2 3 4 5 6

So:

```
sum:    0 1 2 3 4 5 6 7
dp[1]:  0 1 1 1 1 1 1 0
```

Why?

Because with one die:

```
sum 1 → {1}      → 1 way
sum 2 → {2}      → 1 way
sum 3 → {3}      → 1 way
...
sum 6 → {6}      → 1 way
sum 7 → impossible → 0
```

8. Now calculate two dice

We want:

dp[2][7]

Try every possible value of the second die:

```
Second die = 1
Remaining = 7 - 1 = 6
dp[1][6] = 1
```

```
Second die = 2
Remaining = 7 - 2 = 5
dp[1][5] = 1
```

```
Second die = 3
Remaining = 4
dp[1][4] = 1
```

```
Second die = 4
Remaining = 3
dp[1][3] = 1
```

```
Second die = 5
Remaining = 2
dp[1][2] = 1
```

```
Second die = 6
Remaining = 1
dp[1][1] = 1
```

Therefore:

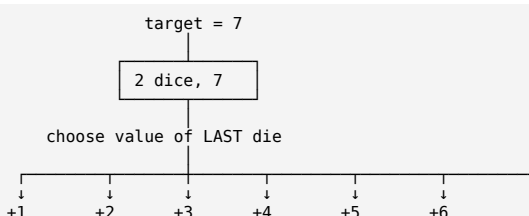
```
dp[2][7]
= 1 + 1 + 1 + 1 + 1 + 1
= 6
```

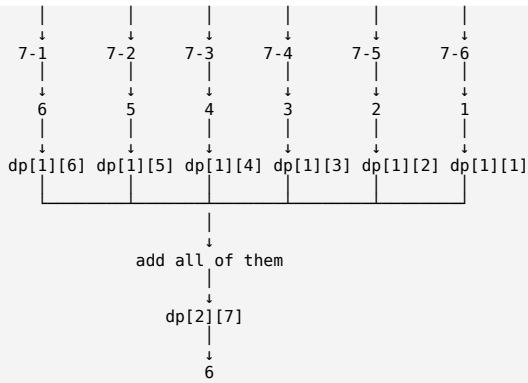
Answer:

6

9. Visualize the transition

The whole idea is:





That's essentially the entire algorithm.

10. Why DP works

Imagine:

```
dp[10][100]
```

There could be thousands/millions of different dice combinations that eventually produce this state.

But we don't care about the individual combinations anymore.

We only need:

How many ways can 10 dice produce sum 100?

And that can be calculated from already-computed smaller states:

```
dp[9][99]
dp[9][98]
dp[9][97]
...
```

So instead of recalculating the same subproblems, we store them.

That's the **overlapping subproblems** property of DP.

11. Bottom-up tabulation

The table is built like this:

	SUM							
	0	1	2	3	4	5	6	7 ...
0	1	0	0	0	0	0	0	0 ...
1	0	1	1	1	1	1	1	0 ...
2	0	0	1	2	3	4	5	6 ...
3	0	0	0	1	3	6	10	15 ...

DICE

For example:

```
dp[2][7] = 6
```

because:

```
dp[1][6] + dp[1][5] + ... + dp[1][1]
= 1 + 1 + 1 + 1 + 1 + 1
= 6
```

And:

```
dp[3][7]
```

would use:

```
dp[2][6]
dp[2][5]
dp[2][4]
dp[2][3]
dp[2][2]
```

```
dp[2][1]
```

12. Important optimization: impossible sums

There is another useful observation.

With `dice` dice:

Minimum possible sum

Every die gives at least 1.

So:

```
minimum = dice
```

Maximum possible sum

Every die gives at most `k`.

So:

```
maximum = dice * k
```

Therefore:

```
if sum < dice
    impossible
if sum > dice * k
    impossible
```

For example:

```
3 dice, k = 6
```

Possible sums are only:

```
3 → 18
```

So:

```
sum = 2 → impossible
sum = 19 → impossible
```

This can reduce unnecessary work.

13. Recurrence

The final recurrence is:

```
dp[dice][sum] =
    dp[dice-1][sum-1]
    + dp[dice-1][sum-2]
    + ...
    + dp[dice-1][sum-k]
```

or mathematically:

```
dp[d][s] = Σ dp[d-1][s-face]
```

for:

```
face = 1 ... k
```

provided:

```
s - face >= 0
```

14. Complexity

There are approximately:

```
n × target
```

DP states.

For each state, we try:

```
k
```

faces.

Therefore:

```
Time = O(n * target * k)
```

With constraints:

```
n <= 30
target <= 1000
k <= 30
```

that's at most roughly:

```
30 * 1000 * 30
= 900,000
```

operations, which is easily manageable.

Space:

```
O(n * target)
```

if we use the full 2D table.

The intuition you should remember

Don't memorize the code. Remember this question:

"What could the last die be?"

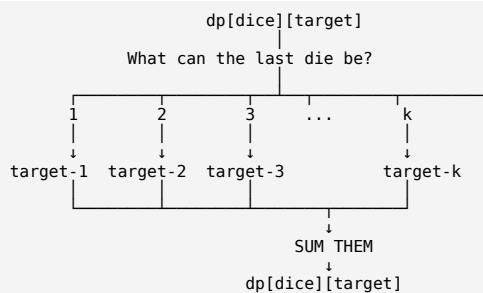
For:

```
dp[dice][target]
```

the last die can be:

```
1 → previous dice need target - 1
2 → previous dice need target - 2
3 → previous dice need target - 3
...
k → previous dice need target - k
```

Therefore:



That is the entire DP logic.

If you understand **"remove the last die → solve the remaining sum with dice-1 dice"**, you have the core intuition for this problem.

Yes. Let's convert the exact logic into **C++ tabulation code**, step by step.

1. DP definition

We said:

```
C++
dp[dice][sum]
```

means:

Number of ways to get **sum** using exactly **dice** dice.

So the answer is:

```
C++
dp[n][target]
```

2. Base case

With zero dice, there is exactly **one way to make sum 0**:

```
C++
dp[0][0] = 1;
```

3. Transition

For every number of dice and every possible sum:

```
C++
for (int dice = 1; dice <= n; dice++) {
    for (int sum = 1; sum <= target; sum++) {
        for (int face = 1; face <= k; face++) {
            if (sum - face >= 0) {
                dp[dice][sum] += dp[dice - 1][sum - face];
            }
        }
    }
}
```

The important line is:

```
C++
dp[dice][sum] += dp[dice - 1][sum - face];
```

It means:

If the current/last die shows **face**, then the remaining **dice-1** dice must produce **sum-face**.

4. Complete code

```
C++
class Solution {
public:
    int numRollsToTarget(int n, int k, int target) {
        const int MOD = 1e9 + 7;
        // dp[dice][sum]
        vector<vector<int>>> dp(n + 1, vector<int>(target + 1, 0));

        // Base case:
        // 0 dice can make sum 0 in exactly 1 way.
        dp[0][0] = 1;

        // Build the DP table
        for (int dice = 1; dice <= n; dice++) {
            for (int sum = 1; sum <= target; sum++) {
                // Try every possible value of current die
                for (int face = 1; face <= k; face++) {
                    // Remaining sum must be non-negative
                    if (sum - face >= 0) {
                        dp[dice][sum] =
                            (dp[dice][sum]
                             + dp[dice - 1][sum - face]) % MOD;
                    }
                }
            }
        }

        return dp[n][target];
    }
};
```

5. Understand the three loops

This is the most important part.

```
C++
```



```
for (int dice = 1; dice <= n; dice++)
```

means:

Build answers for 1 die, then 2 dice, then 3 dice...

Then:

```
C++
for (int sum = 1; sum <= target; sum++)
```

means:

For the current number of dice, calculate every possible target sum.

Then:

```
C++
for (int face = 1; face <= k; face++)
```

means:

Try every possible value of the last die.

So the structure is:

```
DICE
↓
1 → 2 → 3 → ... → n
For each dice:
  ↓
  SUM
  ↓
  1 → 2 → 3 → ... → target
For each sum:
  ↓
  FACE
  ↓
  1 → 2 → 3 → ... → k
```

6. Dry run for n=2, k=6, target=7

Initially:

```
dp[0][0] = 1
```

dice = 1

For **sum = 1**:

```
C++
face = 1
dp[1][1] += dp[0][0]
```

Therefore:

```
dp[1][1] = 1
```

For **sum = 2**:

```
face = 1 → dp[0][1] = 0
face = 2 → dp[0][0] = 1
```

Therefore:

```
dp[1][2] = 1
```

Similarly:

```
dp[1][1] = 1
dp[1][2] = 1
dp[1][3] = 1
dp[1][4] = 1
dp[1][5] = 1
dp[1][6] = 1
```

So:

```
sum:      0 1 2 3 4 5 6 7
         -----
1 die:    0 1 1 1 1 1 1 0
```

dice = 2

Now calculate:

```
dp[2][7]
```

Try every face:

```
face = 1
remaining = 7 - 1 = 6

dp[2][7] += dp[1][6]
          += 1
```

```
face = 2
remaining = 5

dp[2][7] += dp[1][5]
          += 1
```

And:

```
face = 3 → dp[1][4] = 1
face = 4 → dp[1][3] = 1
face = 5 → dp[1][2] = 1
face = 6 → dp[1][1] = 1
```

Therefore:

```
dp[2][7]
= 1 + 1 + 1 + 1 + 1 + 1
= 6
```

So:

```
C++
return dp[2][7];
```

returns:

```
6
```

The one line you should really understand

Everything revolves around:

```
C++
dp[dice][sum] += dp[dice - 1][sum - face];
```

Read it from right to left:

```
dp[dice - 1][sum - face]
    ↓
  remaining dice
    ↓
  remaining sum
    ↓
  add those ways
    ↓
dp[dice][sum]
```

For example:

```
C++
dp[2][7] += dp[1][7 - 3];
```

means:

If the second die is 3, then the first die must make 4.
There is `dp[1][4]` ways to do that, so add those ways to `dp[2][7]`.

That's the entire implementation.

Time: $O(n \times \text{target} \times k)$

Space: $O(n \times \text{target})$

Absolutely. The **memoization version** follows the same recurrence, but instead of building the table with loops, we use **recursion + memo array**.

1. State

We'll define:

```
C++
solve(dice, target)
```

as:

Number of ways to get **target** sum using **dice** remaining dice.

So the final answer is:

```
C++
solve(n, target)
```

2. Recurrence

For the current die, try every face:

```
C++
solve(dice, target)
=
solve(dice - 1, target - 1)
+
solve(dice - 1, target - 2)
+
...
+
solve(dice - 1, target - k)
```

In code:

```
C++
for (int face = 1; face <= k; face++) {
    ways += solve(dice - 1, target - face);
}
```

3. Base cases

If we've used all dice:

```
C++
dice == 0
```

Then:

```
C++
target == 0 → 1 way
target != 0 → 0 ways
```

So:

```
C++
if (dice == 0)
    return target == 0;
```

4. Memoization code

```
C++
class Solution {
public:
    const int MOD = 1e9 + 7;

    int solve(int dice, int target, int k,
              vector<vector<int>>& dp) {
        // No dice left
        if (dice == 0) {
            return target == 0;
        }

        // Impossible target
```

```

    if (target < 0) {
        return 0;
    }

    // Already calculated
    if (dp[dice][target] != -1) {
        return dp[dice][target];
    }

    long long ways = 0;

    // Try every possible face of current die
    for (int face = 1; face <= k; face++) {
        ways += solve(
            dice - 1,
            target - face,
            k,
            dp
        );
        ways %= MOD;
    }

    return dp[dice][target] = ways;
}

int numRollsToTarget(int n, int k, int target) {
    vector<vector<int>> dp(
        n + 1,
        vector<int>(target + 1, -1)
    );

    return solve(n, target, k, dp);
}
};

```

5. Understand the recursion

For:

```

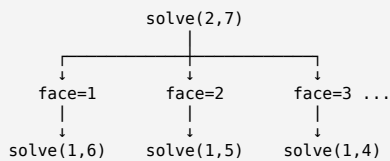
n = 2
k = 6
target = 7

```

we start:

```
solve(2, 7)
```

The current die can be:



Continue:

```

solve(2,7)
= solve(1,6)
+ solve(1,5)
+ solve(1,4)
+ solve(1,3)
+ solve(1,2)
+ solve(1,1)

```

For example:

```
solve(1,6)
```

tries the final die:

```

face = 1 → solve(0,5)
face = 2 → solve(0,4)
face = 3 → solve(0,3)
face = 4 → solve(0,2)
face = 5 → solve(0,1)
face = 6 → solve(0,0)

```

Only:

```
solve(0,0) = 1
```

Therefore:

```
solve(1,6) = 1
```

Similarly:

```
solve(1,5) = 1
solve(1,4) = 1
...
solve(1,1) = 1
```

Thus:

```
solve(2,7)
= 1 + 1 + 1 + 1 + 1 + 1
= 6
```

6. Where is the "memoization" happening?

This part:

```
C++
if (dp[dice][target] != -1) {
    return dp[dice][target];
}
```

checks:

Have I already solved this (**dice**, **target**) state?

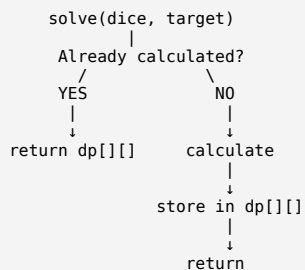
If yes, don't calculate it again.

Then:

```
C++
return dp[dice][target] = ways;
```

stores the answer.

So:



Memoization vs Tabulation

The **logic is exactly the same**.

Memoization

```
C++
solve(dice, target)
```

recursively goes:

```

    solve(n, target)
      ↓
    solve(n-1, ...)
      ↓
    solve(n-2, ...)
      ↓
    base
  
```

Tabulation

We reverse the direction:

```

    dp[0][0]
      ↓
    dp[1][...]
      ↓
    dp[2][...]
  
```

```

...
dp[n][target]

```

So the important thing is:

Memoization = top-down recursion + cache

Tabulation = bottom-up loops + table

The **state and recurrence remain exactly the same**.

Yes — this is the part that usually feels confusing in DP.

The key is: **we are not inventing a new logic for tabulation. We are taking the exact recursion and changing the direction in which states are calculated.**

Let's convert your memoization code to tabulation **mechanically**, step by step.

1. Start with the memoization code

Your important function is:

```

C++
int solve(int dice, int target, int k,
          vector<vector<int>>& dp) {
    if (dice == 0) {
        return target == 0;
    }
    if (target < 0) {
        return 0;
    }
    if (dp[dice][target] != -1) {
        return dp[dice][target];
    }
    long long ways = 0;
    for (int face = 1; face <= k; face++) {
        ways += solve(
            dice - 1,
            target - face,
            k,
            dp
        );
        ways %= MOD;
    }
    return dp[dice][target] = ways;
}

```

The **most important part** is:

```

C++
solve(dice, target)

```

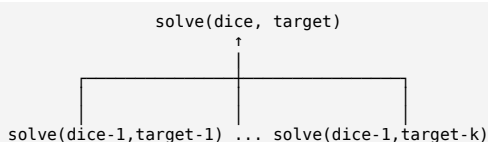
depends on:

```

C++
solve(dice - 1, target - 1)
solve(dice - 1, target - 2)
solve(dice - 1, target - 3)
...
solve(dice - 1, target - k)

```

So:



This dependency tells us **how to build the tabulation table**.

2. First remove the recursion

In memoization, we have:

```
C++
solve(dice, target)
```

In tabulation, we don't want:

```
C++
solve(...)
```

Instead, we will directly access:

```
C++
dp[dice][target]
```

So:

```
C++
solve(dice - 1, target - face)
```

becomes:

```
C++
dp[dice - 1][target - face]
```

This gives us:

```
C++
dp[dice][target] += dp[dice - 1][target - face];
```

This is the exact same recurrence.

3. Now identify the base case

Memoization says:

```
C++
if (dice == 0) {
    return target == 0;
}
```

Meaning:

```
0 dice + target 0 → 1 way
0 dice + target > 0 → 0 ways
```

So our initial DP table should look like:

	sum							
	0	1	2	3	4	5	6	7
dice 0	1	0	0	0	0	0	0	0

Therefore:

```
C++
dp[0][0] = 1;
```

Everything else starts at 0.

This is the **first major conversion**:

MEMOIZATION	TABULATION
if (dice == 0) return target == 0;	dp[0][0] = 1

4. Now look at the recursive dependency

Suppose:

```
dice = 2
target = 7
```

Recursion says:

```
solve(2,7)
├── solve(1,6)
├── solve(1,5)
├── solve(1,4)
├── solve(1,3)
├── solve(1,2)
└── solve(1,1)
```

Notice something very important:

```
solve(2,7)
```

needs:

```
solve(1,...)
```

So **before calculating dice = 2**, we must have calculated **dice = 1**.

That's why our outer loop must be:

```
C++
for (int dice = 1; dice <= n; dice++)
```

Not:

```
C++
for (int target = ...)
```

The dependency tells us the order.

5. First calculate dice = 1

Start with:

```
dp[0][0] = 1
```

Now calculate:

```
dp[1][1]
```

The recurrence says:

```
dp[1][1]
=
dp[0][1-1]
```

Therefore:

```
dp[1][1]
=
dp[0][0]
=
1
```

So:

	0	1	2	3	4	5	6	7
dice = 0	1	0	0	0	0	0	0	0
dice = 1	0	1						

6. Calculate dp[1][2]

The last die can be:

```
face = 1
```

Then previous dice need:

```
2 - 1 = 1
```

So:

```
dp[0][1] = 0
```

Then:


```
face = 2
```

Previous dice need:

```
2 - 2 = 0
```

So:

```
dp[0][0] = 1
```

Therefore:

```
dp[1][2] = 0 + 1 = 1
```

Table:

	0	1	2	3	4	5	6	7
dice = 0	1	0	0	0	0	0	0	0
dice = 1	0	1	1					

7. Calculate dp[1][3]

Try each face:

```
face = 1
→ dp[0][2] = 0

face = 2
→ dp[0][1] = 0

face = 3
→ dp[0][0] = 1
```

Therefore:

```
dp[1][3] = 1
```

Continue.

Because one die can make:

```
1,2,3,4,5,6
```

we get:

	0	1	2	3	4	5	6	7
dice = 0	1	0	0	0	0	0	0	0
dice = 1	0	1	1	1	1	1	1	0

Now **we have completely built the dice = 1 row.**

8. Now calculate dice = 2

This is where the connection to recursion becomes really clear.

Recursion:

```
solve(2,7)
  ↓
solve(1,6)
solve(1,5)
solve(1,4)
solve(1,3)
solve(1,2)
solve(1,1)
```

Tabulation:

```
dp[2][7]
  ↑
dp[1][6]
dp[1][5]
dp[1][4]
dp[1][3]
dp[1][2]
dp[1][1]
```

The recursive calls have simply become **table lookups**.

9. Calculate `dp[2][7]`

We already have:

```
dp[1][1] = 1
dp[1][2] = 1
dp[1][3] = 1
dp[1][4] = 1
dp[1][5] = 1
dp[1][6] = 1
```

Now:

```
dp[2][7]
```

Try every face:

```
face = 1
remaining = 7 - 1 = 6

dp[2][7] += dp[1][6]
```

Then:

```
face = 2
remaining = 5

dp[2][7] += dp[1][5]
```

And:

```
face = 3 → dp[1][4]
face = 4 → dp[1][3]
face = 5 → dp[1][2]
face = 6 → dp[1][1]
```

Therefore:

```
dp[2][7]
=
dp[1][6]
+ dp[1][5]
+ dp[1][4]
+ dp[1][3]
+ dp[1][2]
+ dp[1][1]

= 1 + 1 + 1 + 1 + 1 + 1

= 6
```

10. This gives us the loops

We now know exactly what the loops need to do.

Outer loop

We need to calculate:

```
dice = 1
dice = 2
dice = 3
...
dice = n
```

Therefore:

```
C++
for (int dice = 1; dice <= n; dice++)
```

Second loop

For each number of dice, calculate:

```
sum = 1
sum = 2
...
sum = target
```

Therefore:

```
C++
```

```
for (int sum = 1; sum <= target; sum++)
```

Third loop

For every state, try every face:

```
face = 1
face = 2
...
face = k
```

Therefore:

```
C++
for (int face = 1; face <= k; face++)
```

11. Now literally convert the memoization code

Memoization

```
C++
for (int face = 1; face <= k; face++) {
    ways += solve(
        dice - 1,
        target - face,
        k,
        dp
    );
}
```

Replace recursive call:

```
C++
solve(dice - 1, target - face)
```

with table lookup:

```
C++
dp[dice - 1][target - face]
```

So:

```
C++
for (int face = 1; face <= k; face++) {
    dp[dice][target] +=
        dp[dice - 1][target - face];
}
```

Now **target** isn't a function parameter anymore. It's our loop variable called **sum**.

Therefore:

```
C++
for (int face = 1; face <= k; face++) {
    dp[dice][sum] +=
        dp[dice - 1][sum - face];
}
```

And that's the tabulation transition.

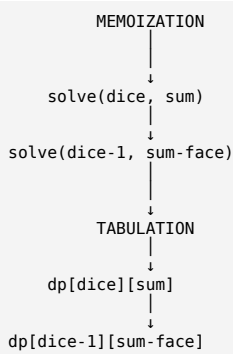
12. Final tabulation code

```
C++
class Solution {
public:
    int numRollsToTarget(int n, int k, int target) {
        const int MOD = 1e9 + 7;
        vector<vector<int>> dp(
            n + 1,
            vector<int>(target + 1, 0)
        );
        // Base case:
        // 0 dice -> sum 0 has exactly 1 way
        dp[0][0] = 1;
    }
};
```

```
// Number of dice
for (int dice = 1; dice <= n; dice++) {
    // Target sum
    for (int sum = 1; sum <= target; sum++) {
        // Value of current/last die
        for (int face = 1; face <= k; face++) {
            if (sum - face >= 0) {
                dp[dice][sum] =
                    (dp[dice][sum]
                     + dp[dice - 1][sum - face])
                    % MOD;
            }
        }
    }
}
return dp[n][target];
};
```

13. The most important conversion

Remember this exact transformation:



So:

```
C++
solve(dice - 1, sum - face)
```

becomes:

```
C++
dp[dice - 1][sum - face]
```

And recursion:

```
solve(...)
↓
solve(...)
↓
solve(...)
```

is replaced by loops:

```
C++
for (dice)
  for (sum)
    for (face)
```

14. Why are there 3 loops?

This is probably the biggest source of confusion.

The **three loops don't represent three different algorithms**.

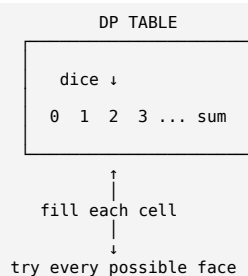
They represent the three things your recursion is varying:

```
solve(
  dice,      ← which dice?
  sum,       ← which sum?
  face       ← which choice?
)
```

Therefore:

```
C++
for (dice)      // which level?
  for (sum)     // which state?
    for (face) // which transition?
```

Think of it this way:



The mental model to remember

When converting **memoization** → **tabulation**, don't think:

"How do I turn recursion into loops?"

Instead think:

"What does my recursive function depend on, and in what order must those dependencies be calculated?"

Here:

```
dp[dice][sum]
  ↓
depends on
  ↓
dp[dice-1][sum-face]
```

So **dice - 1** must be ready **before dice**.

Therefore:

```
C++
for (dice = 1 → n)
```

That is why the tabulation order works.

Memoization: Start at the answer and recursively go toward the base case.

Tabulation: Start at the base case and build toward the answer.

Same states.

Same recurrence.

Same answer.

Only the direction changes.

ChatGPT can make mistakes. Check important info.